

并行与分布式程序设计

绪论

推动并行计算的原因

晶体管集成密度不断提高，但时钟频率的增速急剧放缓

受功耗/散热等限制，单核性能难以提高。

采用并行架构，充分利用资源，多核、众核成为发展趋势。

并行计算的应用

科学仿真、动画渲染、建模

超算

超级计算机（美国Frontier2022性能最强，能效最高 百亿亿次 Pflops/s 我国自研神威太湖之光2016、银河系列、天河系列）

戈登·贝尔奖 中国在戈登·贝尔奖的突破

并行计算软件技术面临的挑战

硬件技术飞速发展，软件生态环境几乎停滞（现有软件难以处理）

硬件发展趋势：众核、异构、每CPU内存...

新软件技术还不成熟，现有代码还未准备好硬件架构的改变

其实软件投资的回报更大

- 足够的并发度（Amdahl定律）
- 并发粒度
 - 独立的计算任务的大小
- 局部性（Cache）
 - 对临近的数据进行计算
- 负载均衡（划分、调度）
 - 处理器的工作量相近
- 协调和同步
 - 谁负责？处理频率？

并行硬件和并行软件

冯·诺依曼结构（SIMD）

主存 内存单元 地址 数据和指令

中央处理器CPU

控制单元：决定执行哪些指令

算术逻辑单元：执行指令

寄存器：CPU中存储数据和程序的特殊快速存储介质

程序计数器：存储要执行的下一条指令的地址

总线：链接CPU和内存的数据、控制、地址电线和硬件

冯诺依曼结构的瓶颈：CPU和主存之间数据传输的速度不匹配

进程是运行着的程序的一个实例，线程包含在进程中

进程 fork 派生线程，join 将线程合并到进程中，线程间的切换比进程间的切换更快

改进冯·诺依曼模型（Cache）

CPU——Cache 缓存：Cache 位于 CPU 同一块芯片或者其他芯片上，CPU 能够快速访问 Cache 缓存

局部性原理：

程序访问完一个存储区域后，往往会访问接下来的区域，这个原理称为局部性。

- 空间局部性 访问临近的区域
- 时间局部性 在不久的将来访问

高速缓存（块）行：

CPU 访问一次内存 **读取一整块代码和数据**，而不只是单条指令或单个数据，这些块称为高速缓存块或高速缓存行。

实际系统中，Cache 通常是多层级的（三级缓存）：

低层 Cache（更快、更小）通常是高层 Cache 的 Cache（高层包含低层）。也有低层不复制高层 Cache 的设计。

Cache 命中：访问的数据在 Cache 中找到，不需要访问主存。

当 CPU 向 Cache 中写数据时，Cache 中的值和主存中的值就会不同或者不一致。

写直达（write-through）Cache：CPU 向 Cache 写数据时，高速缓存行立即写入主存中。

写回（write-back）Cache：数据不是立即更新到主存中，而是将发生数据更新的高速缓存行标记为脏（dirty）。当发生高速缓存行的替换时，标记为脏的高速缓存行写入主存中。

Cache 映射：每个高速缓存行能够放置在 Cache 中的（/唯一位置/n 个位置之一/任意位置）

直接映射：唯一位置

n 路组相连：n 个位置之一

全相联映射：每个高速缓存行能够放置在 Cache 中的任意位置

指令级并行（ILP）

多个处理器或功能单元同时执行指令来提高处理器的性能。

流水线：将**功能单元分阶段安排**

多发射：让**多条指令同时启动**

并行硬件

Flynn’s分类法

SISD单指令单数据流	SIMD单指令多数据流
MISD多指令单数据流	MIMD多指令多数据流

SIMD：数据并行

将数据分配给多个处理器实现并行化，使用相同的指令操作数据子集

SIMD的缺点：

所有ALU要么同时执行相同的指令，要么同时处于空闲状态。

在“经典”的SIMD系统中，ALU必须同步操作。ALU没有指令存储器。

SIMD适用于**大型数据**并行问题，在处理其他并行问题时效果不好。

向量处理器

向量处理器对数组或数据向量进行操作 SIMD使用向量处理器

传统CPU对单独的数据元素或标量进行操作

向量处理器的优点：

速度快，易使用，很高的内存带宽，每个加载的数据都会使用

向量编译器擅长识别向量化的代码，还能提供为什么不能向量化的原因

向量处理器的缺点：

不能处理不规则的数据结构和其他并行结构

可扩展性低。可扩展性是指能够处理更大问题的能力

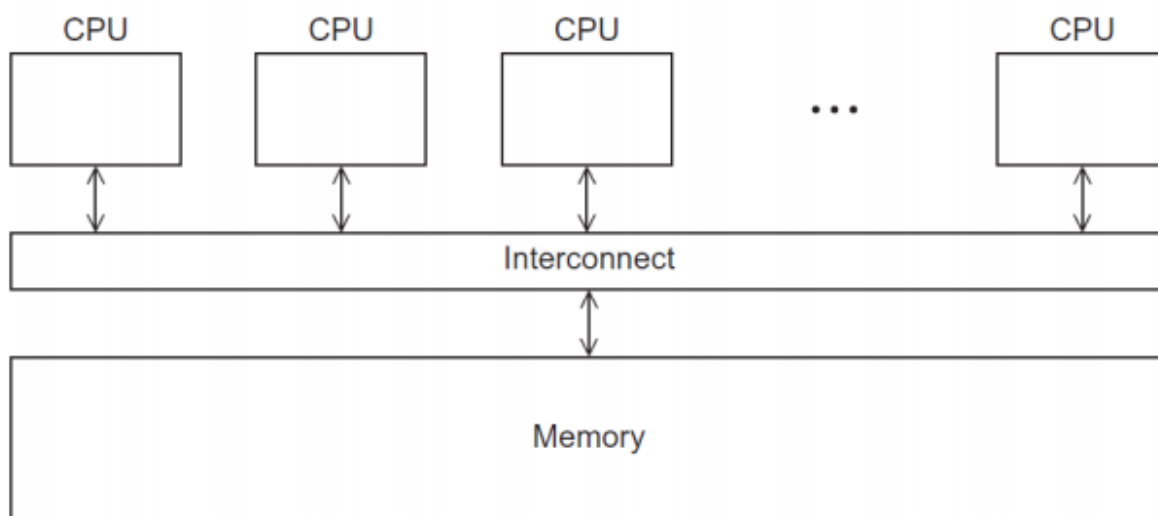
现在所有的GPU都使用SIMD并行（尽管不是纯粹的SIMD并行系统）

MIMD

支持多个指令流在多个数据流上操作

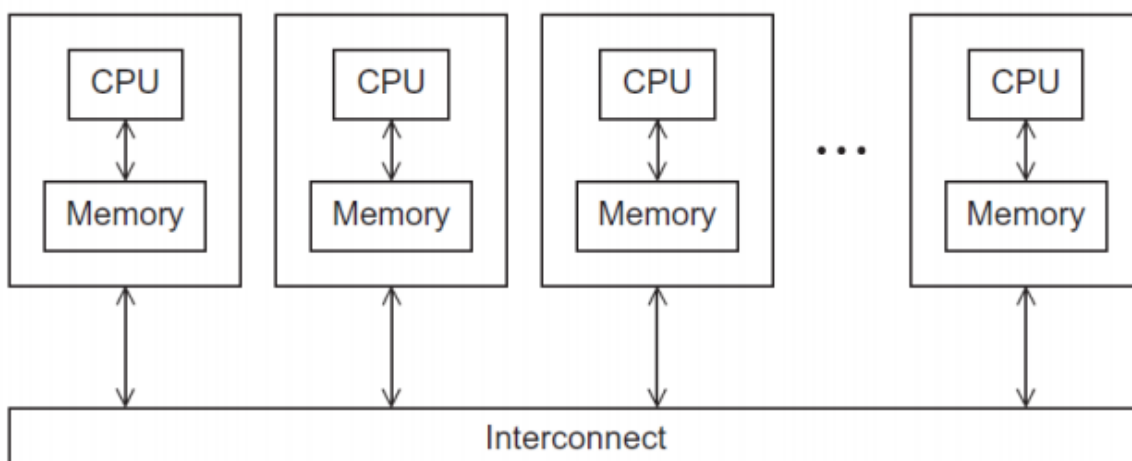
共享内存系统

(一块芯片上有多个CPU、核)



分布式内存系统

(集群 系统中的节点是通过通信网络相互连接的独立计算单元)



互连网络

共享内存互连网络

总线互连:

总线是链接CPU和内存的数据、控制、地址电线和硬件

总线的核心特征是连接到它的设备共享通信线路

随着连接到总线的设备数量的增加，对总线的使用的竞争会增加，性能会下降。

交换互连:

使用交换器来控制相互连接设备间的数据传递。

- 交叉开关矩阵 (Crossbar)
 - 允许不同设备之间同时进行通信。（多个处理器同时访问主存）
 - 比总线速度快。
 - 交换器和链路的开销也更高

分布式内存互连网络

直接互连：

每个交换器与一个（处理器——内存）对直接相连，交换器之间也互相连接。

- 等分带宽
 - 去除最少的链路数，将节点等分成两份
 - 等分宽度是基于最坏情况来估计的
- 带宽
传输数据的速度。兆位每秒、兆字节每秒
- 等分带宽
衡量网络的质量。
不是计算连接两个等分之间的链路数，而是计算链路的带宽。
如果在环中，链路的带宽是10亿位每秒，等分带宽就是20亿位每秒（环中等分带宽为2）

全相连网络（不切实际的）

每个交换器都和其他的所有交换器直接连接。

等分宽度 = $p/4$

间接互连：

交换器不一定与处理器直接连接。

Cache 一致性

程序员无法直接控制缓存及其更新时间。

监听Cache一致性协议

基于目录的Cache一致性协议

并行软件

动态线程：fork派生 join合并

静态线程：线程池

线程安全：各个线程都可以正确执行，不会出现数据污染等

并程序设计的复杂性

- 足够的并发度（Amdahl定律）
- 并发粒度
 - 独立的计算任务的大小
- 局部性（Cache）
 - 对临近的数据进行计算
- 负载均衡（划分、调度）
 - 处理器的工作量相近
- 协调和同步

- 谁负责？处理频率？

并行算法额外开销

除了串行算法要做的之外的工作

线/进程创建、协调、合并

线/进程间通信：最大开销，大部分并行算法都需要

线/进程空闲：负载不均、同步操作、不能并行化的部分
额外计算

并行算法性能评价

看四个例题

加速比：最优串行算法耗时/并行算法耗时

并行算法额外开销： $T_0 = p \cdot T_p - T_s$

加速比： $S = T_s / T_p$

阿姆达尔定律：

除非一个串行程序的执行几乎全部都并行化，否则不论多少可以利用的核，通过并行化所产生的加速比都会是受限的。

$$S = 1 / (1 - a + a / p)$$

□ a 为串行程序中可被(完美)并行化的比例

□ $TS = 1$ ， $TP = T_{\text{不可并行}} + T_{\text{可并行}} = 1 - a + a/p$

□ 加速比的极限是： $S = 1 / (1 - a)$

效率：

度量有效计算时间

$$\square E = S / p = TS / (p \cdot TP)$$

□ 理想情况=1，正常0~1。

□ 因为理想情况 $S = p$

可扩展性：

可扩展性是指能够处理更大问题的能力。

若某并行程序核数(线程数/进程数)固定，并且输入规模也是固定的，其效率值为 E 。现增加程序核数(线程数/进程数)，如果在输入规模也以相应增长率增加的情况下，该程序的效率一直是 E (不降)，则称该程序是可扩展的。

□ 我们希望，保持问题规模不变时，效率不随着线程数的增大而降低，则称程序是可扩展的（称为强可扩展的）。但这往往是难达到的。

□ 退求其次：问题规模以一定速率增大，效率不随着线程数的增大而降低，则认为程序是可扩展的（称为弱可扩展的）。

SIMD向量编程

SIMD概念

单指令多数据

用向量处理器对一组数据（又称“数据向量”）中的每一个元素执行相同的操作 来实现空间上的并行性

（适合应用）SIMD的特点：

- 数据项在内存中连续存储
- 短数据类型：8、16、32位
- 时间局部性，流式数据处理（数据流重用）
- 有时可用来提高计算效率
 - 很多常量
 - 循环迭代短

为什么采用SIMD：

- 更大的并发度、更小的芯片尺寸、设计简单（重复功能的单元）、显示接触硬件（编译器或程序员）

SIMD并行的问题

循环向量化

可向量化的循环：

```

for (i=0; i<100; i+=4)
    A[i+0] = A[i+0] + B[i+0]
    A[i+1] = A[i+1] + B[i+1]
    A[i+2] = A[i+2] + B[i+2]
    A[i+3] = A[i+3] + B[i+3]

```



```

for (i=0; i<100; i+=4)
    A[i:i+3] = A[i:i+3] + B[i:i+3]

```

可部分向量化的循环：

```

for (i=0; i<16; i+=2)
    L = A[i+0] - B[i+0]
    D = D + abs(L)
    L = A[i+1] - B[i+1]
    D = D + abs(L)

```



```

for (i=0; i<16; i+=2)
    L0 = A[i:i+1] - B[i:i+1]
    L1 = A[i:i+1] - B[i:i+1]
    D = D + abs(L0)
    D = D + abs(L1)

```

额外开销

线程间通信是最大开销

- 打包/解包开销：重排数据使之内存连续
 - 打包源运算对象——拷贝到连续内存区域
 - 解包目的运算对象——拷贝回内存
- 对齐开销：调整数据访问，使之对齐
 - 地址是否是向量长度的倍数决定了是否对齐（例如**16字节**）
 - 静态调整循环对齐
 - 动态对齐

- 有时对齐开销会完全抵消SIMD的并行收益
- 控制流导致额外开销
 - 永远是两个控制流路径都执行 (if else)
 - 程序存在控制流时一般不适合使用SIMD

SSE/AVX编程

SSE: Streaming SIMD Extension

AVX: Advanced Vector eXtension

SSE指令对应C/C++ intrinsics

头文件 `#include <emmintrin.h>`

整数 (16字节、8 short、4 int、2 long long、1 dqword)

单精度浮点数 (4 floats) 四个单精度

双精度浮点数 (2 doubles) 两个双精度

PD: 两个双精度, PS: 四个单精度

u

l

h

intrinsic函数命名特点:

- 第一部分: 前缀_mm, 表示是SSE指令集对应的Intrinsic函数。
- 第二部分: 对应的指令操作, 如 `add`, `mul`, `_load`等, 有些操作可能会有修饰符, 如 `loadu` 将16位未对齐的操作数加载到寄存器中。
- 第三部分为操作的对象名及数据类型

如 `_mm_mul_epi32` 对参数中所有的32位有符号整数进行乘法运算

矩阵乘法

Pthread多线程编程

基本概念

POSIX Thread:

- 并发、同步、非显示通信（因为共享内存是隐式的——共享数据的指针传递给线程）

Pthread是POSIX标准

- 相对低层
 - 程序员控制线程管理和协调
 - 程序员分解并行任务并管理任务调度
- 可移植性较好，开发较慢
- 在系统级代码开发中广泛使用，也用于某些类型的应用程序

OpenMP是新标准

- 高层编程，适用于共享内存架构上的科学计算
 - 程序员在较高层次上指出并行方式和数据特性，并指导任务调度
 - 系统负责实际的并行任务分解和调度管理
- 多种架构相关的编译指示

并程序设计的复杂性

- 足够的并发度（Amdahl定律）
- 并发粒度
 - 独立的计算任务的大小
- 局部性
 - 对临近的数据进行计算
- 负载均衡
 - 处理器的工作量相近
- 协调和同步
 - 谁负责？处理频率？

基础API

头文件

- `#include <pthread.h>`

启动线程 pthread_create

```
int pthread_create( pthread_t * thread_id, /out/  
const pthread_attr_t* thread_attribute, /in/  
void * (* thread_fun)(void ), /in*/  
void * fun_arg /in/ );
```

- thread_id 是个pthread_t类型的指针，指向线程ID或句柄（可用于控制停止线程等），须在调用前分配好内存空间
 - thread_attribute 各种属性，通常用空指针NULL表示标准默认属性值
 - thread_fun 新线程要运行的函数（参数和返回值类型都是void）
void可强制转换为任意指针类型，如 long my_rank = (long) rank
 - fun_arg 传递给要运行的函数thread_fun的参数，必须是void类型
- pthread_create 生成并运行了函数：void thread_fun(void* fun_arg)*
- pthread_create 若成功，返回0；若出错，返回非0出错编号

同步

循环步之间的求和运算存在依赖——线程间依赖

可以重排顺序，因为加法运算满足结合律

取数-加法-存结果必须是原子操作，以保持结果与串行执行一致

原子性：一组操作要么全部执行要么全不执行，则称其是原子的。即：不会得到部分执行的结果。

互斥：任何时刻都只有一个线程在执行

临界区：是一个更新共享资源的代码段，一次只能允许一个线程执行该代码段

竞争条件

执行结果依赖于两个或更多事件的时序，则存在竞争条件（race condition）

多个进程/线程尝试更新同一个共享资源时，结果可能是无法预测的，则存在竞争条件。

当多个进程/线程都要访问共享变量或共享文件等共享资源时，如果至少其中一个访问是更新操作（写操作），那么这些访问就可能导致某种错误，称之为存在竞争条件。

数据依赖

数据依赖就是两个内存操作的序，为了保证结果的正确性，必须保持这个序

同步

同步在时间上强制使各执行进程/线程在某一点必须互相等待，确保各进程/线程的正常顺序和对共享可写数据的正确访问

OpenMP共享内存编程

OpenMP是Pthread的升级版，更简单，但限制更多

- 通过少量编译指示指出并行部分和数据共享
- OpenMP能
 - 程序员只需将程序分为串行和并行区域，而不是构建并发执行的多线程
 - 隐藏栈管理 提供同步机制
- OpenMP不能
 - 自动并行化 确保加速 避免数据竞争

OpenMP是Fork-join并行执行模型

- 执行起始是单进程（主线程）
- 并行结构开始
 - 主线程创建一组子线程（工作线程）
- 并行结构结束
 - 线程组同步——隐式barrier
- 只有主线程继续执行

PThread

- 全局作用域变量是共享的
- 栈中分配的变量是私有的

OpenMP

- shared变量是共享的
- private变量是私有的
 - **默认是shared**
 - 循环迭代变量是private

基础API

```
#include <omp.h>
```

- 头文件

```
int omp_get_num_threads(void);
```

- 线程数

```
int omp_get_thread_num(void);
```

- 线程编号 (0到omp_get_num_threads()。主线程编号为0)

```
#pragma omp parallel num_threads(thread_count)
```

- 多线程并行

```
#pragma omp critical
```

- 每个时刻限制只有一个线程执行 (访问临界区)

归约 reduction(+:sum)

归约：相同的规约操作符重复地应用到操作数序列 来得到一个结果的计算。 (**可交换**)

```
sum = 0;
```

```
#pragma omp parallel for reduction(+:sum) 使用for并行循环，并规约
```

```
for (i=0; i < 100; i++) {
```

```
sum += array[i];
```

```
}
```

```
#pragma omp parallel num_threads(thread_count) reduction(+: global_result) 没有使用for 并行循环
```

并行循环(parallel for)

```
#pragma omp parallel for
```

```
for (i=0; i < numPixels; i++) {}
```

创建线程，在线程间分配循环步 (不是所有的for循环均可使用parallel for)

- **迭代次数可预测**
- 不检查依赖性!
- 不支持while、do-while、循环体包含break等

同步 (默认自动使用barrier)

隐式barrier

-并行结构开始和结束位置

其他控制结构的结束位置

用nowait子句可去除隐式同步

显式同步

□ critical

□ atomic (单一语句)

□ barrier

竞争条件和数据依赖

竞争条件 (race condition)

- 执行结果依赖于两个或更多事件的时序

数据依赖 (data dependence)

- 两个内存操作的序，为了保证结果的正确性，必须保持这个序
 - 如果两个内存访问指向相同的内存位置，且其中一个写操作，则它们产生数据依赖

重排转换 (改变顺序使其可并行化)

- 并行化
重排**执行顺序**，保持代码中的依赖关系，则它是安全的
- 局部性优化
修改**访问顺序**，利用cache，保持了代码中的依赖关系，则它是安全的
- 归约计算
对于满足**交换律和结合律的归约操作**，对其重排是安全的

循环调度

负载均衡，调度通信的开销

schedule子句确定如何在线程间划分循环

- static([chunk])静态划分
 - 分配给每个线程 [chunk]步迭代，所有线程都分配完后继续循环分配，直至所有迭代步分配完毕
 - 默认[chunk]为ceil(iterations/threads) 任务数/线程数
- dynmaic([chunk])动态划分
 - 分给每个线程[chunk]步迭代，一个线程**完成任务后**再为其分配[chunk]步迭代
 - 逻辑上形成一个任务池，包含所有迭代步
 - 默认[chunk]为1
- guided([chunk])动态划分，但划分过程中[chunk]指数减小

- 类似于DYNAMIC调度，但分块开始大，随着迭代分块越来越少，循环区间的划分是基于类似下列公式完成的（不同的编译系统可能不同）

默认调度（静态调度）：

`schedule(static, n/thread_count)`

动态：# `pragma omp parallel for num_threads(thread_count) schedule(dynamic, 50)`

Single和Master结构

□线程组中只有一个线程执行代码

□用于I/O或初始化等任务

□入口或出口无隐式barrier

#`pragma omp single {`

`}`

□类似的，只有主线程执行代码

#`pragma omp master {`

`}`

梯形积分

MPI消息传递编程

MPI 基本原语

消息传递是超级计算机和集群主要的编程模型

隔离了独立地址空间

- 没有数据竞争，但可能有通信错误
- 暴露了执行模型，迫使程序员思考局部性，两点对性能都有好处
- 编程复杂，代码膨胀

MPI所有通信、同步都需调用函数完成（无共享变量）

通信

- 点对点通信：消息从进程A发送到进程B
- 组通信（多处理器参与）
 - 移动数据：广播、散发/收集
 - 计算并移动数据：归约、全归约

同步

- 默认使用barrier障碍
- 无锁机制，因为没有共享变量需要保护

查询

- 多少个进程？哪个是我？有处于等待状态的消息？

编译：mpicc -g -Wall -o mpi_hello mpi_hello.c

启动：mpiexec -n 4 ./mpi_hello

进程数

- `int MPI_Comm_size(MPI_Comm comm, int *size)`

进程编号（从0~size-1）

- `int MPI_Comm_rank(MPI_Comm comm, int *rank)`

MPI初始化和结束处理

- `MPI_Init()`
初始化工作,为消息缓冲区分配空间、为进程指定进程号等
`int MPI_Init(
int* argc_p /* 输入/输出参数 /,
char *** argv_p / 输入/输出参数 */);`
- `MPI_Finalize()`
告诉MPI程序已结束，进行清理工作
`int MPI_Finalize();`

MPI 阻塞通信

Send-receive其实完成了两件事

- 数据传输
- 同步

MPI阻塞发送

- `int MPI_Send(void *buf, int count, MPI_Datatype datatype, int dest, int tag, MPI_Comm comm)`
 - `MPI_Send( A, 10, MPI_DOUBLE, 1, ...)`
 - 消息缓冲区 buf count datatype

- 目的进程dest——目的进程在 comm 指定的通信域中的 进程编号
- 阻塞发送
 - 函数返回时，数据已经转给系统发送，缓冲区空闲可另作他用
 - 消息可能还没有发送到目的进程

MPI阻塞接收

- int MPI_Recv(void *buf, int count, MPI_Datatype datatype, int source, int tag, MPI_Comm comm, MPI_Status *status)
 - MPI_Recv(B, 20, MPI_DOUBLE, 0, ...)
 - 源进程——source可以是comm中的编号，或MPI_ANY_SOURCE
 - 标签——tag为特定标签（需匹配）或MPI_ANY_TAG
 - 阻塞接受
 - 等待：直至收到匹配的source和tag发来的消息，缓冲区可另作他用
 - 状态信息——status包含更多信息（如接收到的消息大小）

通信域

- 进程组+上下文
- MPI_COMM_WORLD：默认通信域，其进程组包含所有初始进程

类型匹配

- 宿主语言的类型和消息所指定的类型相匹配
 - 例外：MPI_BYTE、MPI_PACKED
- 发送方和接收方的消息类型相匹配
 - 有类型数据的通信，发送方和接收方均使用相同的数据类型
 - 无类型数据的通信，发送方和接收方均以MPI_BYTE作为数据类型
 - 打包数据的通信，发送方和接收方均使用MPI_PACKED作为数据类型

MPI Tags

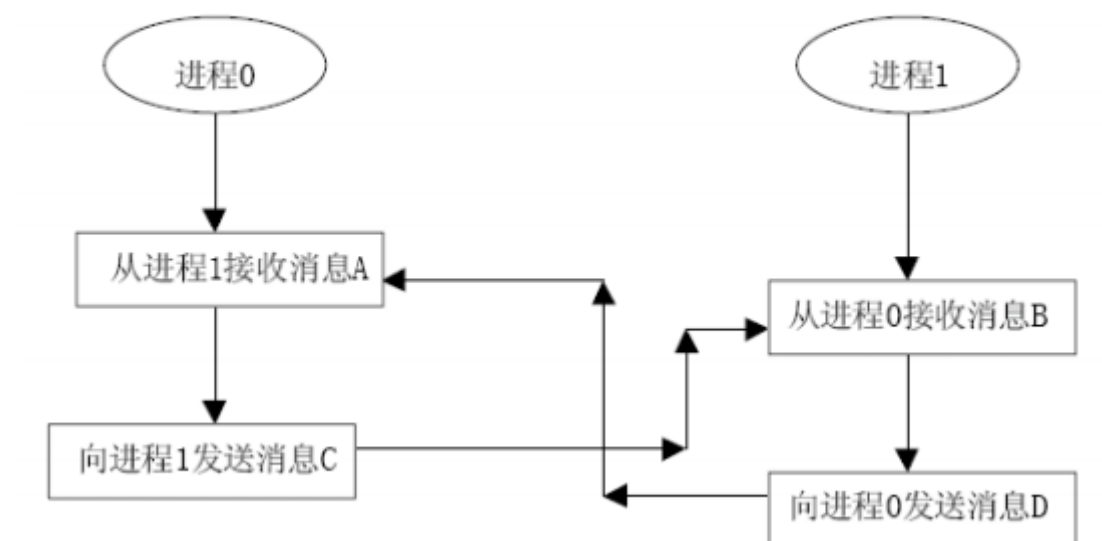
- 发消息时加上标签tag，便于接收方识别筛选消息
- 接收方通过特定tag筛选消息，或指定MPI_ANY_TAG 表示不筛选（接收所有发来的消息）

两个进程都先收后发，必然死锁

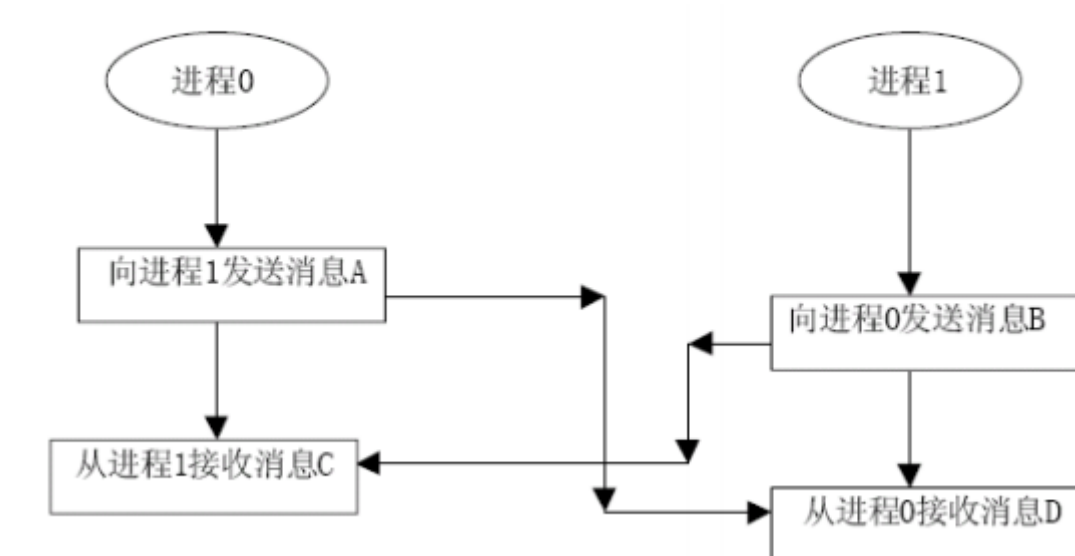
两个进程都先发后受，缓冲区溢出可能会导致死锁

一发一收是安全的

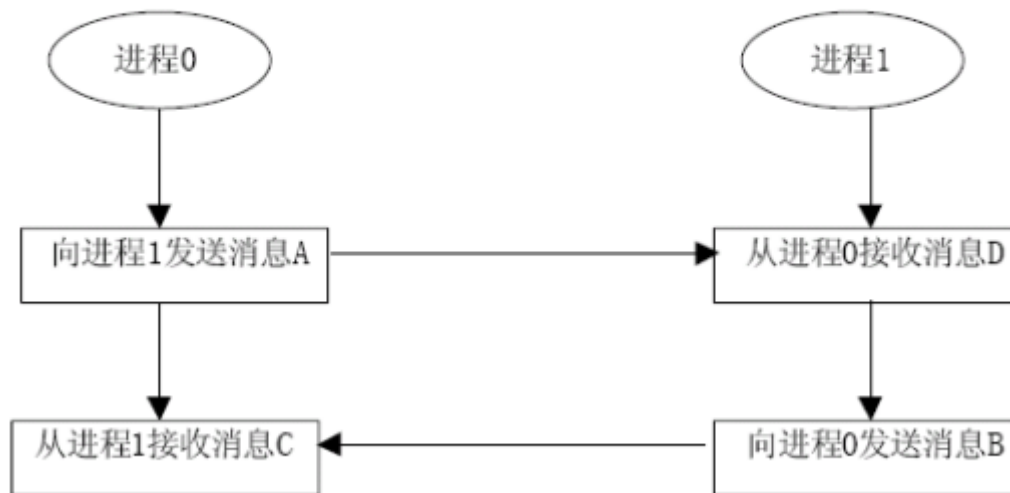
○ 必然死锁的通信模式



○ 缓冲区问题可能导致死锁



安全的通信模式



MPI 编程模型

- 对等式（地位平等，功能相近）
- 主从式（地位不同，功能不同）

组通信

一个进程组（通信域）内的所有进程同时参加通信

所有参与进程的函数调用形式完全相同

哪些进程参加以及组通信的上下文都是由调用时指定的通信域限定的

在组通信中不需要通信消息标志参数

三个功能：通信、同步和计算

操作是成对的——互为逆操作

广播和归约

one-to-all broadcast

□ 一个进程向其他所有进程发送相同数据

□ 初始，只有源进程有一份m个字的数据

广播操作后，所有进程都有一份相同数据

□ 对应操作：all-to-one reduction

□ 初始，每个进程都有一份m个字的数据

□ 归约操作后，p份数据经过计算（加、乘、...）

得到一份数据（结果），传送到目的进程

□ 应用：矩阵相乘、高斯消去、最短路径、向量内积

int MPI_Bcast(void* buffer,int count,MPI_Datatype datatype,int root, MPI_Comm comm)

□ root: 每个调用广播操作的进程中root都相同

□ count: 待广播的数据的个数

MPI归约

int MPI_Reduce(void* sendbuf, void* recvbuf, int count, MPI_Datatype datatype,MPI_Op op, int root, MPI_Comm comm)

环/线性阵列

- 简单算法
 - 源进程顺序将数据发送给其它p-1个进程
 - 低效，源进程成为瓶颈，网络利用不充分
- 递归加倍法
 - 第一步：源进程P→进程P'
 - 第二步：进程P, P'→另外两个进程...
 - 避免冲突，环的实现
 - 第一步：源进程P→距离最远 ($p/2$) 的进程
 - 第二步：两个进程→距离 $p/4$ 的进程...
- 递归加倍法（递归归约法）
- mesh广播算法
- 超立方广播算法
- 归约和广播互为逆过程

All-to-All广播：所有进程同时发起一个广播，每个进程向所有其他进程发送m个字

All-to-All归约：

组收集（All-to-All广播）

□ int MPI_Allgather(void* sendbuf, int sendcount, MPI_Datatype sendtype,void* recvbuf, int recvcount, MPI_Datatype recvtype,MPI_Comm comm)

□ MPI_Allgatherv

□相当于组内每个进程都执行一次收集

归约并散发（All-to-all归约）

int MPI_Reduce_scatter(void* sendbuf, void* recvbuf, int *recvcounts MPI_Datatype datatype, MPI_Op op, MPI_Comm comm)

□ 归约后再执行一次散发操作

线性阵列和环

□保证每个节点总有数据传输给邻居——

链路总保持忙

□第一步：每个进程→某个邻居进程

□第二步：每个进程将刚收到的数据转发到下一进程...

算法思想：更有效地利用链路：p个all-to-all广播同时进行，一个链路上同时传输的消息合并为大消息

非阻塞通信

调用返回≠通信完成

计算和通信的重叠

非阻塞发送和非阻塞接收

多线程混合编程

MPI是进程（独立地址空间）间并行

线程并行是进程内的共享内存模型

纯MPI：节点内和跨节点都采用MPI实现进程并行，节点内MPI内部是采用共享内存来通信的

两种混合编程：

MPI + OpenMP：节点内使用OpenMP，跨节点使用MPI

MPI + Pthreads：节点内使用Pthreads，跨节点使用MPI

CUDA编程

GPU SIMD SPMD

CUDA编程模型

host: 指代CPU端的代码 (主机)

device: 指代GPU端的代码 (设备)

kernel: 从host调用, 在device端运行的函数

Serial Code (host)

Parallel Kernel (device)

KernelA<<< nBlk, nTid >>>(args);

线程层次结构

内存层次结构

CUDA内存层次简介

- Device代码可以
 - 读/写每线程独占的寄存器
 - 读/写每kernel共享的全局内存
- Host代码可以
 - 在CPU主存和Device全局内存间传输数据

Host-Device内存数据传输API:

- `cudaMemcpy ( void * dst, const void * src, size_t count, cudaMemcpyKind kind );`

优化方式 (掩盖访存开销、利用shared memory)

(掩盖访存开销、利用shared memory)

问题所在:

- 1. M和N均需要被从global中获取WIDTH次
- 2. 每从global中取一次, 只计算一次加法和乘法
- 即访存/计算=1:1
- 解决方法:
- 使用shared memory!

shared变量的地址只能在device代码中使用

掩盖访存延迟 分片

- 将指令发送给warp中所有线程需要4个时钟周期
- **一次访存可执行n个指令**
- 至少需要 $500/4n$ 个warp掩盖延迟

连续32个线程组成一个warp

在SM上运行

□ SM

: Stream Multiprocessor,

流多处理器

□ Warp

是GPU执行程序时的调

度单位

, 同一个Warp里的线

程执行相同的指令